General info:

and = 1st falsy
or = 1st truthy

Lists

- Lists: useful functions
  - lst.append(value) # add value to end of list → whole thing as object
  - lst.extend(lst2) # add all values from lst2 to end of lst → must be list
  - lst.insert(index, value) # insert value at given index
  - lst.remove(value) # removes first instance of given value
  - lst.pop(index) # removes & returns value at given index → list.pop() = lst.pop(last value)
  - lst[i:j] # creates sub list from index i to j-1 → not same list
- Lists: List comprehensions
  - [<map exp> for <name> in <iter exp> if <filter exp>] (if <filter exp> optional)
  - [<map exp> if <filter exp> else <alternate map exp> for <name> in <iter exp>]

[1:] → copy of list, w/ everything but 1st item
[:1] → [0:1] selects first item
[:-1] → whole list minus last item

HOFs

- function that takes in another function as an input (Function as Arguments)
- function that returns another function as an output (or both) (Function as return val)
- functional list operations
  - Goal: transform list, return new result
  - map/filter/reduce (function, list_of_inputs)

list(map(func_to_apply, list_of_inputs))
list(filter(condition, list_of_inputs))
reduce(function, list_of_inputs)

Function	Action	Input Arguments	Input Fn. Returns	Output
map	Transform every item	1 (each item)	"Anything", a new item	List: same length, possibly new values
filter	return a list w/ fewer items	1 (each item)	a Boolean	List: possibly less items, same values
reduce	"combine items together"	2 (current item & previous result)	type matches type of each item	a "single" item

Environment Diagrams

- Creating a function
  1. draw function signature
  2. function parent is frame where it's defined
  3. If we used def, bind the name to the value in the current frame
- Calling user-defined functions
  - operator → operands → apply operator to operands
  - create new frame
  - bind formal parameters to evaluated operands
  - evaluate function body
  - after you're done, return to the frame that called the function
    - (no explicit RV, return None)
- Assignment → assigning variables
  - evaluate expressions on the right-hand side
  - bind that value to the left-hand side

ADT

- Constructors
  - create an instance of ADT using HOF & given data structure
  - takes in as arguments some attributes of our ADT
  - returns a representation of the object (i.e list or dict)
- Selectors (attributes)
  - access elements of the instance of the ADT
  - use it to not break abstraction barrier
- other functions related take in representation created by constructor & do smth w/ it
- Abstraction barrier
  - violated if: not using selector (i.e using x[:], assuming list or dict)

Dictionaries

- creating & populating
  - d = {1: 'a', 2: 'b', 3: 'c'}
- creating then populating
  - d = {}
  - d[1], d[2], d[3] = 'a', 'b', 'c'
- values: anything
- keys: immutable & unique (can't change value) (≠ list, dict, func)
  (can be int/float/str/bool, tuples)
- dictionary Methods
  - x in dict → checks if key x is in dict
  - dict[x] → returns value paired w/ key x
  - for k in dict → iterates through the keys (k) of a dictionary
  - dict.pop(x) → removes key x from dict and returns its value
  - dict.get(x[, default]) → same as dict[x] but default included → return default if x not in dict
  - dict.keys() → returns an iterable of keys in dict
  - dict.values() → " " of values in dict
  - for key, value in dict.items() → iterates over keys & values in dict

Recursion

- Base case: simplest possible input
  - "simplest solution to problem?"
  - ex: 0, None, [], ""
  - can have multiple
- Recursive case: continually breaking down problem into simpler subproblem
  - calling recursive function on a smaller input
  - each call gets input closer to base case
  - "How to call recursive function onto smaller input?" (i.e factorial on n-1)
  - "How to use subproblem solutions to get final solution?"
    - assume recursive call gives correct answer (i.e n × factorial of (n-1) bcuz 5! = 5·4!...)
- Solve:
  - build off recursive result to get final answer
- Tree Recursion:
  - multiple recursive calls

WWPD questions

①
```
def example():
    print('Do Nothing')
    return None
x = 8
y = 88
z = 'python'
def w(d):
    x = 0
    while x > 0:
        return None
    return d + 1
def fun(f, x, y):
    f = min
    return f(x, y)
```

x * y ⟶ 704

example() ⟶ Do Nothing
None

x or y ⟶ 8 ← first truthy

(x and y) * (x/y) ⟶ 8.0
↑ first falsy

x * (x > y) ⟶ Error
8 * 8 > 88
False

'Py' + z ⟶ 'Pypython'

fun(max, 61, 88) ⟶ 61
↓ min

```
lam = lambda x: lambda y: w(x)
lam(2)(3)
```
→ 3 (2+1)

②
```
>>> x = 5
>>> y = 4
>>> z = [lambda y: y, lambda x: x, lambda x: y]*
>>> def g(L):
        x = 6
        return L(x)
>>> r = lambda n: lambda phi: z[-2](phi(n))
```
not callable

- z[1] ⟶ Function
- [q(x) for q in z] ⟶ [5,5,4] *
- [x + n for n in z] ⟶ Error
- g(lambda y: x + 1) ⟶ 6
- r(1)(2) ⟶ Error

③
```
>>> 'young' and 'sweet' or 'seventeen' → 'sweet'
>>> see = ['see', 'that', 'girl', 'watch', 'that', 'scene']
    see[1:3] → ['that', 'girl']
>>> see[6] ⟶ Error
>>> digging = lambda x: lambda: 17 + y
    digging ⟶ Function
>>> digging() ⟶ Error
```
missing x

⑤
```
>>> lucky = (lambda x: lambda y: (y-x) % 8 == 0)(x)
>>> def mystery(x, y):
        if lucky(x):   lucky(40) → y = 40
            print('ready')
        while x > y:
            x = x // 10
            print(x)
        return 'go'
>>> print('a', mystery(7, 9))
```
lucky(7) y = 7

a go

```
>>> mystery(40, 2)
'ready'
4
0
'go'
```

④
```
def f1(x, y):
    if x > (x+y):
        print(x)
        y = x
    return x+y
def f2(a, b):
    if a:
        return b and a
    else:
        return a or b
f3 = lambda lst: lst[1:] + lst.pop(0)
```
if x > (x-y): print(y); x = y

after 1

```
>>> f1(3, -5)
3
3
6
>>> f1(-4, 7)
7
14
>>> f2(print('a'), 10 % 3)
a
1
```
when you first call it

```
>>> f3([[10, 20], 30, 40])
[30, 40, 10, 20]
```

④ sum_to_single_digit:
```
""" 12345 → 15 → 6, tallied 2"""
def sum_to_single_digit(num):
    tally = 0
    while num > 9:          ← not 1 digit
        print(num)
        digit_sum = 0
        while num > 0:
            digit_sum += num % 10      → right digit
            num = num // 10            → left digits
        num = digit_sum
        tally += 1
    print(num)
    return tally
```

⑤ def falling(n,k):
```
""" falling factorial of n to depth k
    falling(6,3) → 120 # 6*5*4 """
total = n
a = 1
if k == 0:
    return 1
while (k-1) > 0
    k = k-1
    total = total * (n-a)
    a = a+1
return total
```

⑤ WWPD (SP 2023 Final)
```
func = lambda x, y: add(mystery(x), y)
def add(x, y):
    return x+y
def cond(x):
    return x < (lambda y: y/x)(18)
def mystery(x):
    arr = [1, 4, 5, 2, 3]
    return (lambda x: (arr[-1]) * x))(x)
    x += 1
```
func = lambda x, y: add(mystery(x), y) → add(mystery(4), -2) → add(12, -2)

cond(9) → 9 < (lambda y: y/x)(18) → 9 < 2 ?

mystery(0) → 0 division Error

mystery(4) → 18 18 → 3 → 0 → 0 → 4 → 4

- mystery(0) ⟶ 0
- mystery(2) ⟶ 6
- cond(0) ⟶ Error
- cond(9) ⟶ False
- func(4, -2) ⟶ 10

⑥
```
def savage():
    yield 5000
    yield from [400, 500]
megan = savage()
L = list(megan)
def merge(rene, gade):
    i1 = iter(rene
    i2 = iter(gade)
    try:
        while True:
            yield next(i1)
            yield next(i2)
    except StopIteration:
```
```
>>> print(print('2020', print('fun')), megan)
fun
2020 None
None Iterator
>>> L
[5000, 400, 500]
>>> next(megan)   ↳ bic all done
Error
>>> actors
['Cruise', 'Hanks', 'Diesel', 'Hanks', 'Diesel']
>>> list(merge(artists, actors))
```

Recursion Questions

①
```
def factorial(n):
    if n == 1:                      # 1! = 1   ] base
        return 1
    else:                           # recursive
        return n * factorial(n-1)   # same func on smaller output
                                    # solve ↑ current number
```

②
```
def is_increasing(n):
    """ >>> is_increasing(255) → False
        is_increasing(12345) → True """
    right_digit = n % 10
    rest = n // 10
    if rest == 0:                   # base
        return True
    elif right_digit <= rest % 10:
        return False
    return is_increasing(rest)
```

③
```
def even_odd(num):
    """ num is alternating even, odd digits & 1st & 3rd ≠
        >>> even_odd(234) → True """
    def even_odd_helper(num, is_even, prev, prev_of_prev):
        if num == 0:                # base
            return True
        last_digit = num % 10
        is_even_curr = last_digit % 2
        if is_even == is_even_curr or last_digit == prev_of_prev:
            return False
        return even_odd_helper(num // 10, is_even_curr, last_digit, prev)
    return even_odd_helper(num, num % 2 == 0, -1, -1)
```

④
```
def exaggerate(words, vowels, repeat):
    """ >>> vowels = ['a','e','i','o','u']
        >>> exaggerate("woah so cool", vowels, 2)
        'Wooaah soo cooool' """
    if words == '':                 # base ← nothing to go through
        return ''                   # returns ''
    else:
        cur = words[0]              # first word
        if cur in vowels:           # if it's in vowel
            cur = cur * repeat
        rest = exagerate(words[1:], vowels, repeat)   # take out first word → less words
        return cur + rest
```

⑤
```
def flip_flop(n):
    """ flip_flop(7315) → 7+3-1+5 = 14
        & get_length(n) → number of digits in n """
    if n < 10:                      # base
        return n
    else:
        last_digit = n % 10
        if get_length(n) % 2 == 0:
            return flip_flop(n // 10) + last_digit
        else:
            return flip_flop(n // 10) - last_digit
```

⑥
```
def sum(n):
    """ sum(5) → 15 # 1+2+3+4+5 """
    if n == 1:
        return 1
    else:
        return sum(n-1) + n
```

⑦
```
def has_seven(k):
    """ >>> has_seven(3) → False
        has_seven(2734) → True
        has_seven(717) → True """
    if k % 10 == 7:                 # right most
        return True
    if k == 0:
        return False
    elif k % 10 != 7:
        return has_seven(k // 10)   # removes right most
```

⑧
```
def hail_stone_iterative(n):
    print(n)
    count = 1
    while n > 1:
        if n % 2 == 0:
            n = n // 2
            print(n)
            count += 1
        else:
            n = n * 3 + 1
            print(n)
            count += 1
    return count


def hailstone_recursive(n):
    print(n)
    if n == 1:                      # base
        return 1
    if n % 2 == 0:
        return hailstone_recursive(n // 2) + 1   # count
    else:
        return hailstone_recursive(n * 3 + 1) + 1
```

⑨
```
def fib(n):
    """ fib(5) → 5 """
    if n < 2:                       # or: if n == 0 or n == 1:
        return n                    # if n == 0: return 0
    return fib(n-1) + fib(n-2)      # elif n == 1: return 1
```

⑩
```
def count_change(value, coins):
    """ >>> denominations = [50, 25, 10, 5, 1]
        >>> count_change(7, denominations) → 8 """
    if value < 0 or len(coins) == 0:    # base
        return 0                        # value < 0, or no coins left
    elif value == 0:
        return 1                        # valid count
    using_coin = count_change(value - coins[0], coins)   # smaller amt of money → use coin
    not_using_coin = count_change(value, coins[1:])      # fewer coins → "discard coin"
    return using_coin + not_using_coin
```

⑪ Product of list
```
def nested_product(lst):
    if not lst:
        return 1
    if isinstance(lst[0], list):
        return nested_product(lst[0]) * nested_product(lst[1:])
    else:
        return lst[0] * nested_product(lst[1:])
```

List Questions

① similarList:

```
def remove_sub(big_list, sub_list)
    '''
      remove_sub([1,2,3,2,3,2,3,7], [2,3])
      >>> [1,7] '''
    curr_index = 0
    result = []
    while curr_index < len(big_list):
        if big_list[curr_index: curr_index + len(sub_list)] == sub_list:
            curr_index += len(sub_list)
        else:
            result.append(big_list[curr_index])
            curr_index += 1
    return result
```

② def shift_right(people):

```
    ''' new list: ["Bob", "Alice", "Jane"] → ["Jane", "Bob", "Alice"] '''
    return [people[-1]] + people[:-1]
```

last person ↑ ↑ everyone, but last person

```
def shift_right_n_times(people, n):
    ''' shift_right_n_times(["Bob", "Alice", "Jane"], 2)
      → ['Alice', 'Jane', 'Bob'] '''
    for i in range(n): ← repeat it
        people = shift_right(people)   ← rename it
    return people
```

③ def aggregate(n, func, cond, base): ← HOF

```
    ''' add_times_three = lambda x,y: (x+y) * 3
      is_even = lambda x: x%2 == 0
      aggregate(122, add_times_three, is_even, 0) → 6   (0+2)*3
      aggregate(4212, add, is_even, 0) → 3   ((0+2)+2)+4
    '''
    result = base
    while n>0:
        digit = n%10
        n = n//10
        if cond(digit) and not digit == n%10:
            result = func(result, digit)
    return result
```

HOFs Questions

① HOF that takes in function:

```
def becomes_even(func, num):
    """ True if func for num is even"""
    return func(num) % 2 == 0
def triple(a):
    return a * 3
x = becomes_even(triple, 2)
```

Global Frame [p=G]
becomes_even → func becomes_even(func, num)
triple → func triple(a) [p=G]
x
f1: becomes_even [p=G]
func → func triple(a) [p=G]
num | 2
rv | True
f2: triple [p=G]
a | 2
rv | 6

② HOF that returns a function:

```
def make_multiplier(x):
    "creates & returns func multiply that mult. input by x"
    def multiply(y):
        return x * y
    return multiply
mul_5 = make_multiplier(5)
a = mul_5(3)
```

Global
make_multiplier → func make_in... (x) [p=G]
mul_5 → func multiply(y) [p=f1]
a | 15
f1: make_multiplier [p=G]
x | 5
multiply
rv
f2: multiply [p=f1]
y | 3
rv | 15

③ map_range:

```
def map_range(start, stop, f):
    """ returns sum of applying f to all
      values from start→stop, inclusive"""
    result = 0
    for i in range(start, stop + 1):
        result += f(i)
    return result
```

⑤ count_cond(mystery_func, n):

```
""" count_cond(divisible, 4) → 3 # 1,2,4 """
i, count = 1, 0
while i <= n:
    if mystery_function(n,i) == True:
        count += 1
    i += 1
return count
```

④ pairwise_apply

```
def pairwise_apply(lst1):
    """ takes in list a, returns func that takes in list b & returns another func,
    that takes in a arg func f & returns output of helper, which returns
    a list by applying f to elements of list a & b w same index.
    >>> pairwise_apply([6,5,3,4])([2,3,2,1])(max) → [6,5,3,4] """
    def helper(lst2, func):
        output = []
        if len(lst1) != len(lst2):        # ensures length is same
            return "ERROR"
        for i in range(len(lst1)):
            output.append(func(lst1[i], lst2[i]))   # append each individual item into list
        return output
    return lambda lst2: lambda f: helper(lst2, f)   # return func back onto same line
```

⑥ def mul_by_num(factor):

```
""" x = mul_by_num(5)
f(3) → 15
y = mul_by_num(2)
y(-4) → -8 """
def func(num):
    return num * factor
return func
```

⑦ Higher order combiners

```
def higher_order_combiners(combiner, start, is_brake):
    """ return one arg funcs until brake reached True & combo of results returned
    c1 = higher_order_combiner(lambda x, y: x+y, 0, lambda x: x % 2 == 0)
    c1(1)(3)(5)(6) #1+3+5+6
    15 """
    def helper(x, combo):
        if is_brake(x):
            return combiner(x, combo)
        else:
            return lambda y: helper(y, combiner(x, combo))
    return lambda x: helper(x, start)
```

⑧ def multi_caller(*functions):

```
def apply(*arguments):
    result = []
    for i in range(len(arguments)):   # means indexing
        func = functions[i]
        args = arguments[i]
        output = func(args[0], args[1])
        result.append(output)
    return result
return apply
```

ADT Questions

① Constructor:

```
def make_boyfriend(name, age, cute):
    return {"name": name, "age": age, "cute": cute}
```
(cute: method)

Selectors:

```
def get_name(boyfriend):
    return boyfriend["name"]
def get_age(boyfriend):
    return boyfriend["age"]
def get_cute(boyfriend):
    return boyfriend["cute"]
```
(can access dict values through its keys)

Operations:

```
def in_age_range(age, boyfriend):
    ''' too old: (age*2)-7, too young: (age/2)+7, no Chris
    her age = 24 '''
    minimum = age/2 + 7
    maximum = 2*age + 7
    bf_age = get_age(boyfriend)
    bf_name = get_name(boyfriend)
    if bf_name != "Chris" and bf_age >= minimum and bf_age <= maximum:
        return True
    return False
```
(and = first "false-y" value; or = first "truth-y" value)

③ Test Distributions

(//10 → gets rid of last #)

```
def dist_dictionary(scores):
    ''' dict. distribution w bins of test scores in 10 & percentaage
    of scores in it '''
    distribution = {}
    for score in scores:
        bin = (score // 10) * 10
        if bin in distribution:        # exists = update
            distribution[bin] += 1
        else:
            distribution[bin] = 1      # add new entry
    for bin in distribution:
        distribution[bin] = distribution[bin] / len(scores)   # how many scores
    return distribution
```

Dictionaries Questions

① Chef's Assistant

```
def add_new_orders(all_orders, new_orders):
    ''' take all_orders dict & mutate to include new orders list
    >>> order = {'fries': 3, 'burger': 4}
    >>> add_new_orders(order, [('fries', 4), ('milk', 2)])
    >>> order == {'fries': 7, 'burger': 4, 'milk': 2}
    True '''
    if len(new_orders) == 0:
        return
    food_item = new_orders[0][0]
    quantity = new_orders[0][1]
    if food_item in all_orders:
        all_orders[food_item] += quantity      # update
    else:
        all_orders[food_item] = quantity       # add new key
    add_new_orders(all_orders, new_orders[1:])
```
(passing in same dict updated; to get remaining elements in list)

② Voting Machine

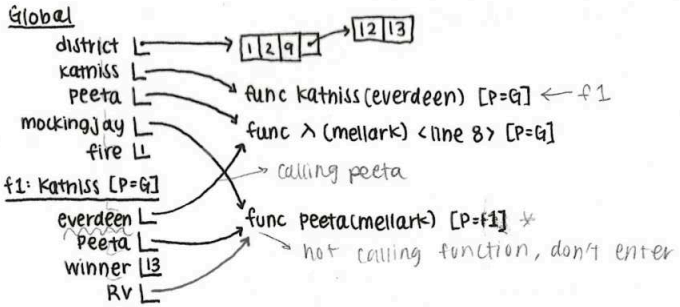
```
def add_votes(name, total, votes):
    '''>>> votes = {'Michael': 400, 'Hetal': 100}
    >>> add_votes('Karim', 75, votes)
    >>> votes = {'Michael': 400, 'Hetal': 100, 'Karim': 75}'''
    if name in votes:
        votes[name] += total      # update
    else:
        votes[name] = total       # new entry
def vote_summary(votes, parties):
    ''' >>> parties = {'Prof Party': ['Michael'], 'TA Party': ['Karim', 'Hetal']}
    >>> votes = {...}
    >>> vote_summary(votes, parties)
    Professor Party: 500
    TA Party: 175 '''
    for party in parties:
        votes_count = 0
        for name in votes:
            if name in parties[party]:
                votes_count += votes[name]
        print(party, ':', votes_count)
```

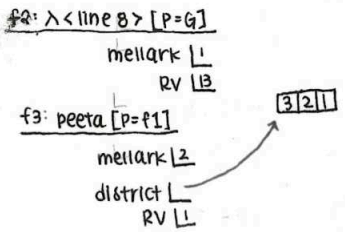
Environment Diagram Questions

① district = [1, 2, 9, [12, 13]]

```
def katniss(everdeen):
    def peeta(mellark):
        district = [3,2,1]
        return district[mellark]
    winner = everdeen(1)
    return peeta
peeta = lambda mellark: district[3][mellark]
mockingjay = katniss(peeta)
fire = mockingjay(2)
```

→ calling a function = new frame 1

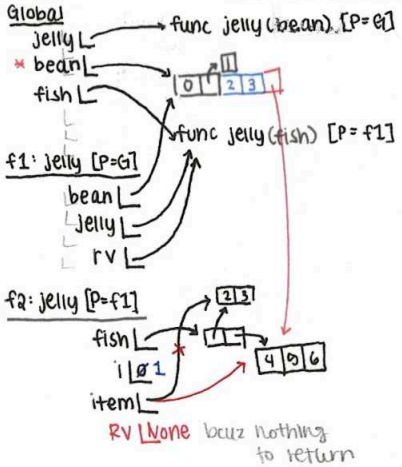




②

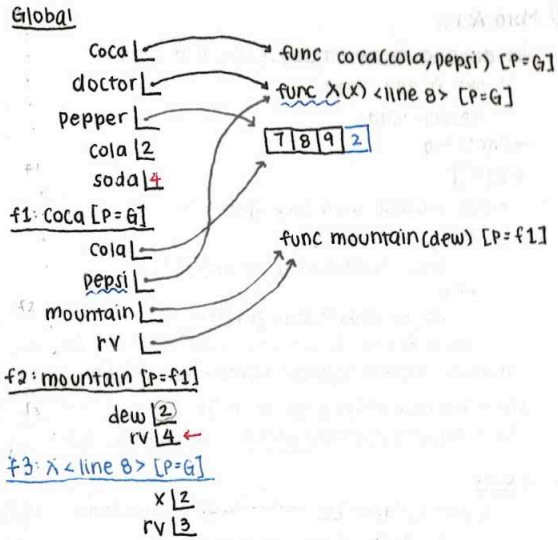
```
def jelly(bean):
    def jelly(fish):
        i=0
        while i< len(fish):
            item= fish[i]
            if len(item) % 2 == 0:
                bean.extend(item)
            else:
                bean.append(item)
            i+= 1
        return jelly
bean = [0, [1]]
fish = jelly(bean)
fish([[2,3], [4,5,6]])
print(len(bean))
```

fish points to jelly(fish) = open new frame



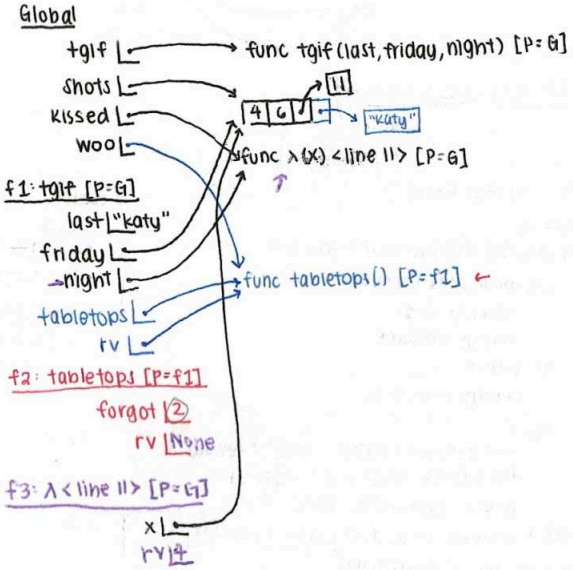
③ ~~Soft~~ Drinks

```
def coca(cola, pepsi):
    def mountain(dew):
        cola.append(dew)
        cola[dew] = pepsi(dew)
        return len(cola)
    return mountain
doctor = lambda x: x+1
pepper = [7,8,9]
cola = 2
soda = coca(pepper, doctor)(cola)
```



④ TGIF

```
def tgif(last, friday, night):
    friday.append([last])
    def tabletops():
        forgot = 2
        if len(friday) >=4:
            friday[forgot].extend([night(friday)])
        else:
            friday.append(night)
    return tabletops
shots = [4,6,[11]]
kissed = lambda x: len(x)
woo = tgif("katy", shots, kissed)
woo()
```



⑤ More Abba

```
def dancing_queen(friday_night, instrument):
    if not friday_night:
        return 'sad'
    lights = 0
    lst = []
    while (friday_night and lights < 3):
        if lights % 2 == 1:
            lst.append([instrument[3]] * 2)
        else:
            lst.append(instrument[1])
        lights += 1 *
    return lambda chance: lights + chance + 68
feel = lambda beat: beat or beat / 0
ohh = dancing_queen(feel(17), 'tambourine')(17)
```

17 → Truthy

0 < 3 → True

Global
- dancing_queen → func dancing_queen(friday_night, instrument) [P=G]
- feel → func λ(beat) <line 14> [P=G]
- ohh

f1: λ <line> [P=G]
- beat 17
- rv 17

f2: dancing_queen [P=G]
- friday_night 17
- instrument 'tambourine'
- lights ~~0 1 2~~ 3
- lst → empty list → 'a' 'a' → 'b' 'b'
- rv → func λ(chance) <line 12> [P=f2]

f3: λ <line 12> [P=f2]
- chance 17
- rv

③ Milk or cereal first?

```
milk = 10
def pour_cereal(amount, first):
    def pour_milk(amount):
        amount -= 1
        return amount
    if first:
        return pour_milk
    else:
        froot_loops = ["red", "blue", "yellow"]
        froot_loops.append(["purple"])
        return pour_milk(amount)
first = lambda x: x and print("yum!")
pour_cereal(2, first(milk))
```

* print returns None

Global
- milk 10
- pour_cereal → func pour_cereal(amount, first, colors) [P=G]
- first → func

f1: λ line 22 [P=G]
- x
- RV None

⑥ Psych! (CSp 2023 Final)

```
theme = ["line", "obscurity", "line"]
def song(maturity):
    if maturity:
        theme.append(["alot", "maturity"])
    else:
        theme.extend(["right", "wrong"])
i = 1
run = lambda crawl: theme[1]
def know(i,u):
    theme = ["any", "proof"]
    if i and u:
        return run(88)
    else:
        return run
return know
deception = song(8 % 4)
bend = deception(-1,1)
psych = len(theme)
```

Global
- theme → "line" "obsc." "line" "right" "wrong"
- song → func song(maturity) [P=G]
- deception
- bend "obs"
- psych 5

f1: song [P=G]
- maturity 10
- i 1
- run → func λ(crawl) <line 8> [P=f1]
- know → func know(i,u) [P=f1]
- RV

f2: know [P=f1]
- i ~~1~~
- u 1
- theme → "any" "proof"
- RV "obs"

f3: λ <line 8> [P=f1]
- crawl 88
- RV "obs"

⑦ Breakfast Time! (CSp 2022 Final)

```
breakfast = ['toast', 'croissant']
confiture = 10
def tartine(jam):
    if jam % 5 == 0
        breakfast[0] += 'jam'
    return hot_drink(8)
def hot_drink(tay):
    breakfast.append(tay)
    return lambda choco: choco + tay
plate = tartine(confiture)(2)
```

Global
- breakfast → 'toast' 'croissant' 8
- confiture 10
- tartine → func tartine(jam) [P=G]
- hot_drink → func hot_drink(tay) [P=G]
- plate 10

f1: tartine [P=G]
- jam 10
- RV → func λ(choco) <line 11> [P=f2]

f2: hot_drink [P=G]
- tay 8
- RV

f3: λ <line 11> [P=f2]
- choco 2
- RV 10